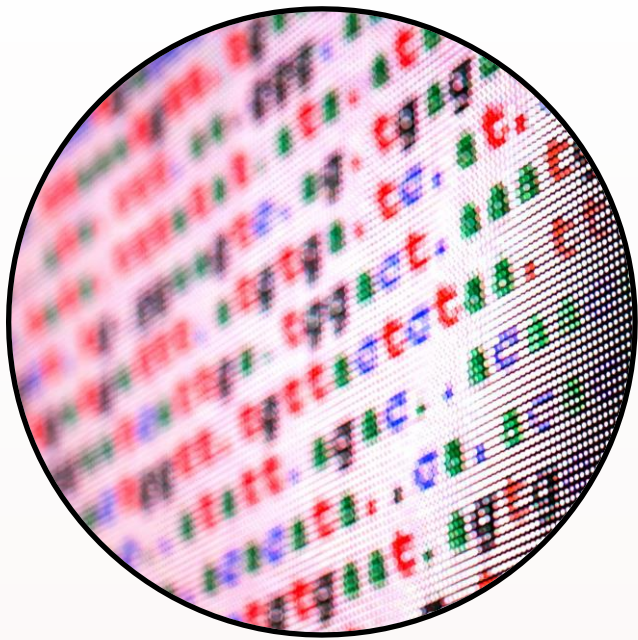




# Ofuscación y protección de datos sensibles

La ofuscación consiste en modificar los datos originales de forma controlada, de manera que sigan siendo útiles para el análisis o la práctica técnica, pero no permitan identificar a personas reales ni exponer información confidencial.

**¿Qué datos deben ofuscarse? En general, datos personales.**

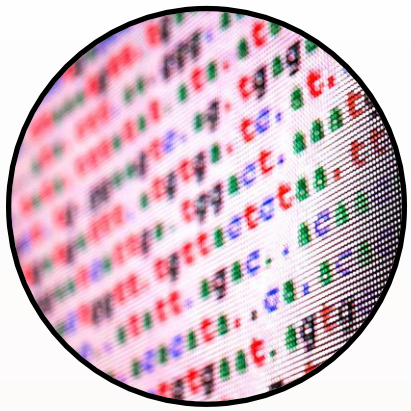
- email
- dni/nif
- telefonos
- nombres
- Direcciones
- otros

**¿Por qué se ofuscan los datos?**

La ofuscación de datos es una práctica fundamental por motivos de:

- **Privacidad** de clientes y empleados
- **Cumplimiento de normativas de protección de datos** (por ejemplo, RGPD)
- **Buenas prácticas de seguridad de la información**
- Uso seguro de datos reales en entornos educativos y de prueba

El objetivo es que los datos **mantengan su estructura y coherencia**, pero **pierdan cualquier vínculo con la identidad real**.



# Ofuscación y protección de datos sensibles

## ¿Cómo se ofuscan los datos?

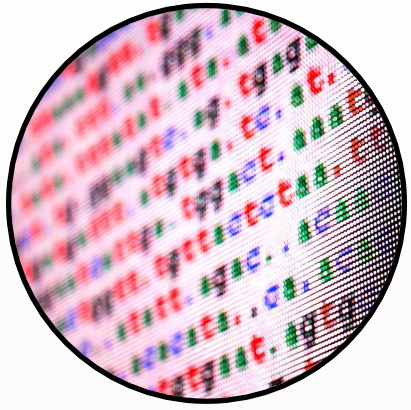
Para realizar la ofuscación se utilizarán **expresiones regulares (regex)**.

Las expresiones regulares son una herramienta que permite trabajar con texto de forma **flexible**, definiendo **patrones** que describen cómo es un determinado **tipo de información**.

De forma sencilla, una regex sirve para decirle al ordenador “qué aspecto tiene” un dato, por ejemplo:

- Un correo electrónico (texto + @ + dominio)
- Un número de teléfono (una secuencia de dígitos)
- Un DNI o NIF (números y letras con un formato concreto)

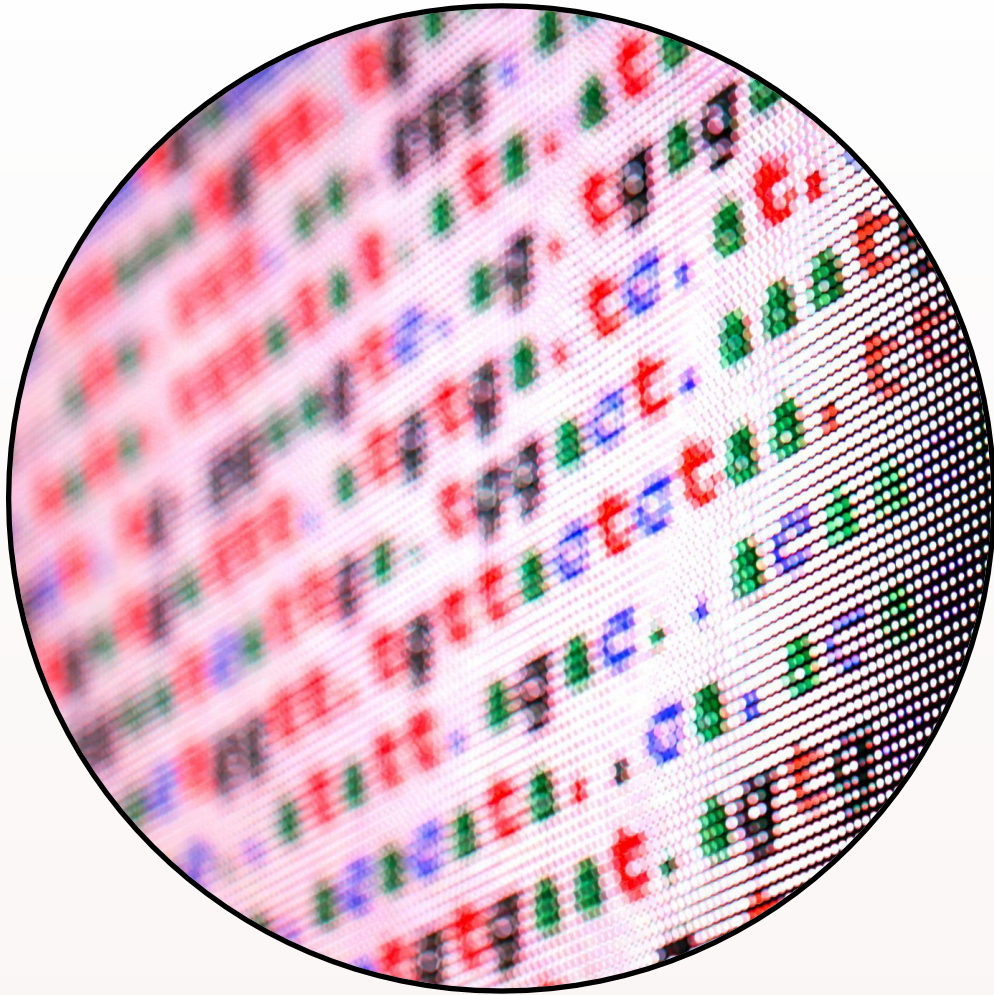
Una vez definido el patrón, podemos **buscar esos valores dentro de un texto** y **reemplazarlos** automáticamente por una etiqueta genérica.



# Ejemplo de ofuscación

**¿Cómo se ofuscan los datos?**

- "maria.lopez@empresa.com" → <email>
- "12345678Z" → <dni>
- "B12345678" → <nif>
- "612345678" → <telefono>
- "María López" → <nombre>
- "Madrid" → <lugar>
- "calle de la oca" → <direccion>
- "250" → <importe>



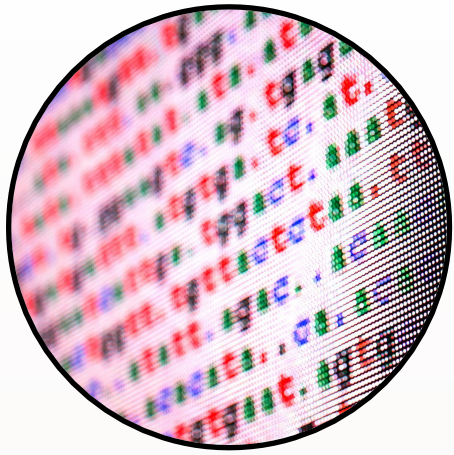
# Expresiones regulares

El módulo **re** de Python es la biblioteca estándar para trabajar con expresiones regulares (RegEx).

Permite **buscar, extraer, dividir y reemplazar texto** basándose en patrones específicos en lugar de coincidencias exactas.

[¿Cómo usar regex con Python?](#)

[Prueba tus expresiones regulares](#)



# Expresiones regulares

## Metacaracteres comunes

| Símbolo | Significado                     | Ejemplo de Patrón | Coincidencia (Match)     |
|---------|---------------------------------|-------------------|--------------------------|
| .       | Cualquier carácter (excepto \n) | c.sa              | "casa", "cosa", "c-sa"   |
| ^       | Inicio de la cadena             | ^Hola             | " <b>Hola</b> mundo"     |
| \$      | Final de la cadena              | fin\$             | "esto es el <b>fin</b> " |
| *       | 0 o más repeticiones            | ba*               | "b", "ba", "baaaaa"      |
| +       | 1 o más repeticiones            | ba+               | "ba", "baaaaa" (no "b")  |
| ?       | 0 o 1 repetición (opcional)     | color?            | "colo", "color"          |
| {n}     | Exactamente <i>n</i> veces      | \d{3}             | "123", "890"             |
| []      | Conjunto de caracteres          | [aeiou]           | Cualquier vocal          |
|         | Operador O (OR)                 | si no             | "si" o "no"              |
| \       | Escapa caracteres especiales    | \.com             | ".com" (literal)         |

## Cuantificadores de repetición

Los cuantificadores \*, + y ? controlan la cantidad de veces que un patrón puede repetirse en la expresión regular.

## Clases y agrupaciones

Los corchetes [] definen clases de caracteres y los paréntesis () agrupan expresiones para capturas y combinaciones.

## Operador alternativo

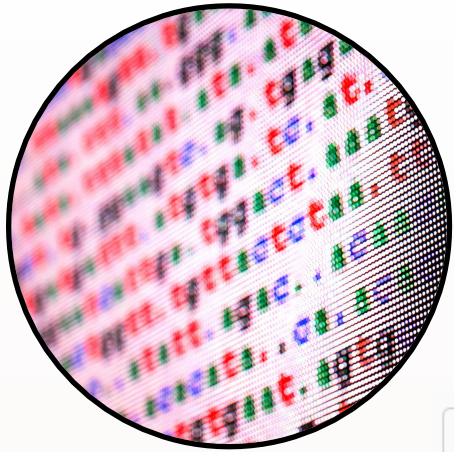
El operador | permite definir alternativas para que el patrón coincida con múltiples opciones posibles.



# Expresiones regulares

Ejemplo de patrones para números:

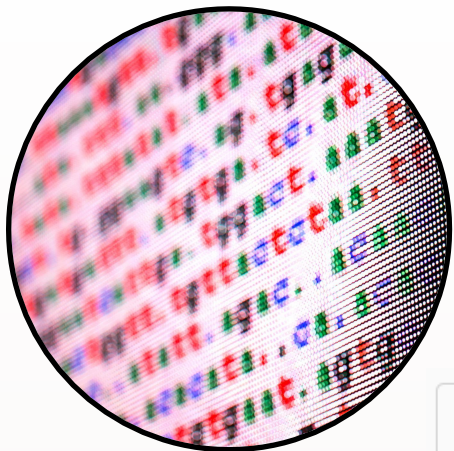
- `\d+` → Extrae números enteros simples (1, 23, 456)
- `-?\d+` → Extrae enteros que pueden ser negativos (-3, 10)
- `\d+\.\d+` → Extrae decimales con punto (3.14, 0.5)
- `\d+,\d+` → Extrae decimales con coma (2,75, 10,50)
- `\d+[.,]\d+` → Extrae decimales con punto o coma (3.14, 3,14)
- `\d+(?:[.,]\d+)?` → Extrae enteros o decimales (45, 3.5, 7,89)
- `[+-]?\d+(?:[.,]\d+)?` → Extrae números con signo (+30, -4.5, 12)
- `\d{1,3}(?:[.,]\d{3})+` → Extrae números con separador de miles (1.234, 12,345, 1.234.567)
- `[A-Z]+\d+` → Extrae códigos alfanuméricos tipo letras+números (ABC123, ID45)
- `(?<!\w)\d+(?!\\w)` → Extrae números aislados, no pegados a letras
- `^\d+` → Extrae números al inicio de línea
- `\d+$` → Extrae números al final de línea
- `\((\d+)\)` → Extrae números que están entre paréntesis ((34), (100))



# Expresiones regulares

Ejemplo de patrones para letras:

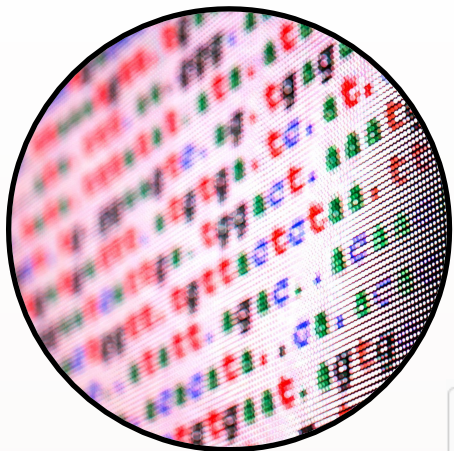
- `[a-z]+` → Extrae solo letras minúsculas (abc, hola, python)
- `[A-Z]+` → Extrae solo letras mayúsculas (ABC, HOLA, PYTHON)
- `[a-zA-Z]+` → Extrae letras sin importar el caso (Hola, Test, abcXYZ)
- `[A-Z][a-z]+` → Extrae palabras tipo Nombre propio (Hola, Marta, Python)
- `[a-zA-Z]{3}` → Extrae secuencias de EXACTAMENTE 3 letras (abc, XYZ)
- `[a-zA-Z]{3,}` → Extrae palabras con 3 o más letras (hola, Python)
- `[a-z]{2,5}` → Extrae minúsculas entre 2 y 5 letras (ho, casa, perro)
- `[A-Z]{2,5}` → Extrae mayúsculas entre 2 y 5 letras (AB, CASA, PRUE)
- `[A-Za-z0-9]+` → Extrae texto alfanumérico (abc123, Test1, X9)
- `[A-Z]{2}\d+` → Extrae códigos tipo “LETRAS+NÚMEROS” (AB123, XY45)
- `[a-z]+(?:\d)` → Extrae minúsculas antes de un número (abc de abc123)
- `(?:\d)[A-Za-z]+` → Extrae letras después de un número (XYZ de 45XYZ)
- `^[A-Za-z]+` → Extrae letras al inicio de línea
- `[A-Za-z]+$` → Extrae letras al final de línea
- `\b[A-Za-z]+\b` → Extrae palabras formadas solo por letras (ignora números y símbolos)



# Expresiones regulares

## Ejemplo de patrones para fechas:

- `\d{2}/\d{2}/\d{4}` → Extrae fechas en formato dd/mm/yyyy (16/02/2026)
- `\d{2}-\d{2}-\d{4}` → Extrae fechas en formato dd-mm-yyyy (16-02-2026)
- `\d{4}-\d{2}-\d{2}` → Extrae fechas en formato ISO yyyy-mm-dd (2026-02-16)
- `\d{4}/\d{2}/\d{2}` → Extrae fechas yyyy/mm/dd (2026/02/16)
- `\d{1,2}/\d{1,2}/\d{2,4}` → Extrae fechas flexibles: días y meses de 1 o 2 dígitos (1/2/2026, 01/2/26)
- `\d{1,2}-\d{1,2}-\d{2,4}` → Igual que arriba pero con guiones (1-2-2026, 01-02-26)
- `\d{1,2}\s+[A-Za-z]+\s+\d{4}` → Extrae fechas tipo “16 febrero 2026” o “1 March 2024”
- `[A-Za-z]+\s+\d{1,2},\s*\d{4}` → Extrae fechas tipo “February 16, 2026”
- `\d{1,2}\s+[A-Za-z]{3}\s+\d{4}` → Extrae fechas abreviadas tipo “16 Feb 2026”
- `[A-Za-z]{3}\s+\d{1,2},\s*\d{4}` → Extrae fechas abreviadas como “Feb 16, 2026”
- `\d{8}` → Extrae fechas compactas sin separadores (16022026, 20260216)
- `(?:\d{2}|\d{4})?:\d{1,2}?:\d{2}|\d{4}` → Extrae fechas con cualquier separador (., /, -)
- `\b\d{1,2}(?:st|nd|rd|th)\s+[A-Za-z]+\s+\d{4}\b` → Fechas en inglés con sufijos (16th February 2026)



# Expresiones regulares

## Ejemplo de patrones para fechas:

- `\d{2}/\d{2}/\d{2}` → Fechas abreviadas con año de 2 dígitos (16/02/26)
- `\d{2}-\d{2}-\d{2}` → Igual pero con guiones (16-02-26)
- `\d{4}\.\d{2}\.\d{2}` → Fechas con punto estilo europeo/ISO (2026.02.16)
- `\d{1,2}\sde\s[A-Za-z]+\sde\s\d{4}` → Formato español literal “16 de febrero de 2026”
- `[A-Za-z]+\s\d{4}` → Mes + año (Febrero 2026, March 2024)
- `\d{4}` → Extrae solo el año (2026, 1998) si lo necesitas en regex amplias
- `(?:Q[1-4])\s\d{4}` → Años por trimestre (Q1 2026, Q4 2025)
- `\d{4}-W\d{2}-\d` → Fecha ISO por semana y día (2026-W07-1)
- `\d{4}-W\d{2}` → Semana ISO (2026-W07)
- `\d{1,2}[/-]\d{1,2}` → Fechas incompletas sin año (16/02, 16-2, 1.2)
- `\d{1,2}\s*(?:jan|feb|mar|apr|may|jun|jul|aug|sep|oct|nov|dec)[a-z]*\s*\d{0,4}` → Fechas con meses abreviados en inglés (16 Feb 2026, 1 Sept)
- `(?:jan|feb|mar|apr|may|jun|jul|aug|sep|oct|nov|dec)[a-z]*\s\d{1,2}` → Variante “mes día” (Feb 16, Sept 1)
- `\d{14}` → Timestamp compacto y sin separadores (20260216143055 = yyyyMMddHHmmss)
- `\d{8}T\d{6}Z` → Timestamp ISO compacto (20260216T143055Z)
- `\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}` → Fecha completa estilo ISO 8601 (2026-02-16T14:30:55)



# Expresiones regulares

Exploración de patrones con **search()**:

Busca el del patrón en cualquier parte de la cadena y devuelve un objeto Match.

```
re.search(pattern, string, flags=0)
```

## Parámetros

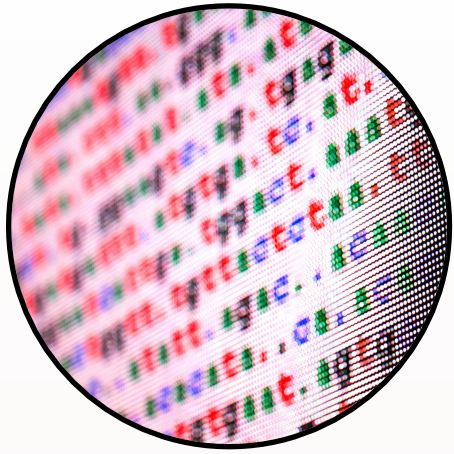
- **pattern** → El patrón (expresión regular) que quieres buscar.
- **string** → La cadena donde se realiza la búsqueda.
- **flags** (opcional) → Modificadores como re.IGNORECASE, re.MULTILINE, etc.

## Qué devuelve

- Un objeto **Match** si encuentra la primera coincidencia.
- None si no encuentra ninguna.

```
import re
```

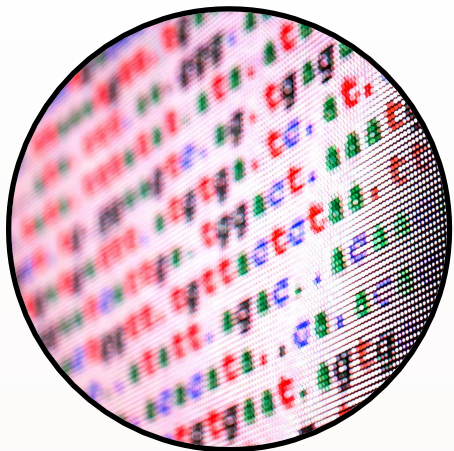
```
m = re.search(r"\d+", "Hoy es 16 de febrero")  
print(m.group()) # → 16
```



# Expresiones regulares

## Objeto Match

| Función            | Descripción                          |
|--------------------|--------------------------------------|
| .group(n)          | Devuelve la coincidencia o un grupo  |
| .groups()          | Devuelve todas las capturas de grupo |
| .groupdict()       | Devuelve grupos nombrados como dict  |
| .start(n) /.end(n) | Índices de inicio/fin                |
| .span(n)           | Tupla con (start, end)               |
| .string            | Cadena original                      |
| .re                | Patrón usado                         |



# Expresiones regulares

Busca la **primera coincidencia** del patrón la cadena y devuelve un objeto Match

Exploración de patrones con **match()**:

```
re.match(pattern, string, flags=0)
```

## Parámetros

- **pattern** → El patrón (expresión regular) que quieres buscar.
- **string** → La cadena donde se busca, solo al inicio.
- **flags** (opcional) → Modificadores como re.IGNORECASE,.

## Qué devuelve

- Un objeto **Match** si encuentra coincidencia al principio.
- None si no encuentra ninguna.

```
import re
```

```
resultado = re.match(r"hello", "hello, world!")
```

```
if resultado:
```

```
    print("Coincidencia encontrada:", resultado.group())
```

```
else:
```

```
    print("No hay coincidencia")
```



# Expresiones regulares

Exploración de patrones con **fullmatch()**: Tiene que coincidir 100% la cadena.

```
re.fullmatch(pattern, string, flags=0)
```

```
re.fullmatch(r"\d{3}-[A-Z]{2}", "123-AB")    # ✓ Match
re.fullmatch(r"\d{3}-[A-Z]{2}", "123-AB-9")  # ✗ None
```

## Qué devuelve

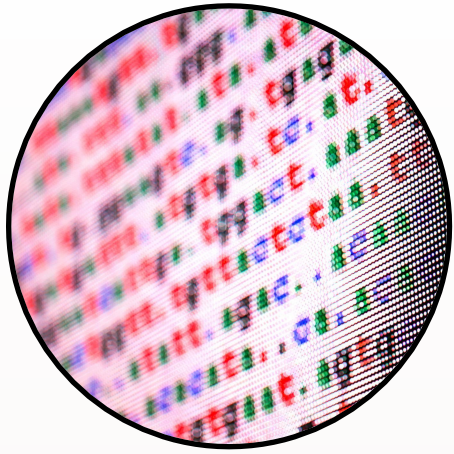
- Un objeto **Match** si encuentra coincidencia al principio.
- None si no encuentra ninguna.

Exploración de patrones con **findall()**:

```
re.findall(pattern, string, flags=0)
```

Retorna todas las apariciones del patrón.

```
re.findall(r"\d+", "Hay 3 gatos y 14 pájaros")
# → ['3', '14']
```



# Expresiones regulares

Exploración de patrones con **sub()**:

Reemplaza todas las partes de una cadena que coinciden con un patrón

```
re.sub(pattern, replacement, string, count=0)
```

```
xto = "ID=123 y valor=45"  
resultado = re.sub(r"\d+", "###", texto)  
# Mi número es ###-###-###
```

Exploración de patrones con **split()**:

Divide una cadena usando un patrón regex como separador

```
re.split(pattern, string, maxsplit=0)
```

```
texto = "a, b;   c       d"  
partes = re.split(r"[;,]\s*|\s{2,}", texto)  
# ['a', 'b', 'c', 'd']
```



# Expresiones regulares

Exploración de patrones con **compile()**:

```
re.compile(pattern, flags=0)
```

## ✓ Beneficios:

- Mayor eficiencia cuando vas a usar el mismo patrón muchas veces.
- Código más limpio y más fácil de leer.
- Permite usar métodos del patrón compilado:  
.match(), .search(), .findall(), .sub(), etc.

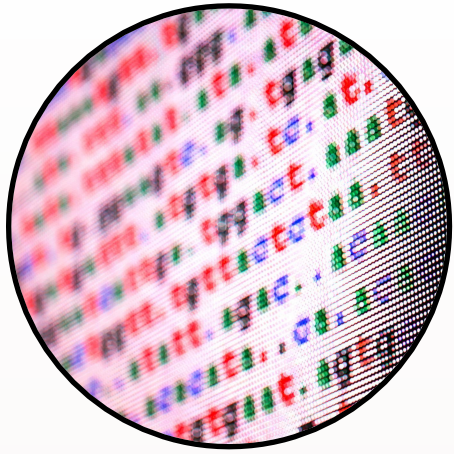
Crea un objeto patrón (regex compilado) a partir de una expresión regular.

Ese objeto se puede reutilizar varias veces, evitando recompilar la misma regex en cada llamada.

```
# Compilamos una regex para números
pat = re.compile(r"\d+")

print(pat.findall("Hay 3 perros y 14 gatos"))
# → ['3', '14']

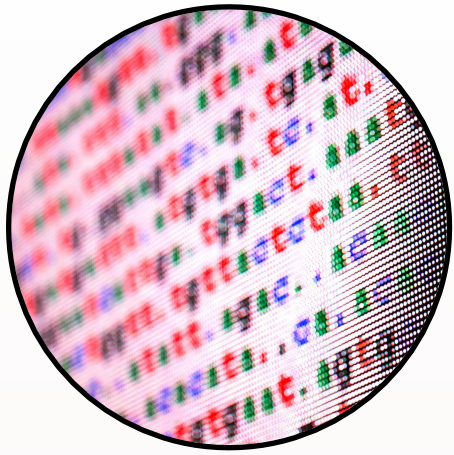
m = pat.search("abc123xyz")
print(m.group())
# → '123'
```



# Expresiones regulares

## Básicos

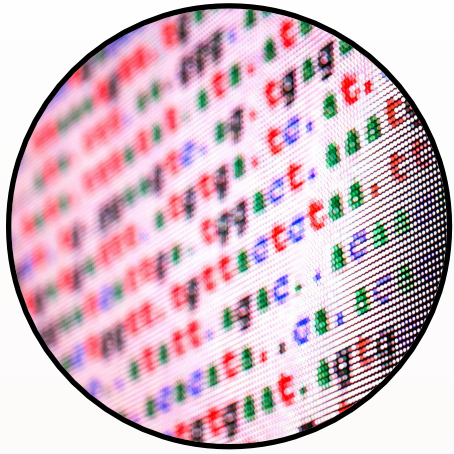
| Metacarácter | Ejemplo de uso                     | Resultado | Descripción                               |
|--------------|------------------------------------|-----------|-------------------------------------------|
| \d           | Patrón: \d+ sobre "Tengo 12 gatos" | ['12']    | Coincide con dígitos (0–9)                |
| \w           | Patrón: \w+ sobre "A1_bC"          | ['A1_bC'] | Letras, números o guion bajo              |
| \s           | Patrón: \s+ sobre "Hola mundo"     | [' ']     | Espacios, tabuladores o saltos            |
| .            | Patrón: a.c sobre "abc"            | ['abc']   | Cualquier carácter excepto salto de línea |
| [A-Z]        | Patrón: [A-Z]+ sobre "ABC123"      | ['ABC']   | Rango de letras mayúsculas                |
| [^0-9]       | Patrón: [^0-9]+ sobre "abc123"     | ['abc']   | Coincide con todo excepto dígitos         |



# Expresiones regulares

## Agrupadores, anclas y lookarounds

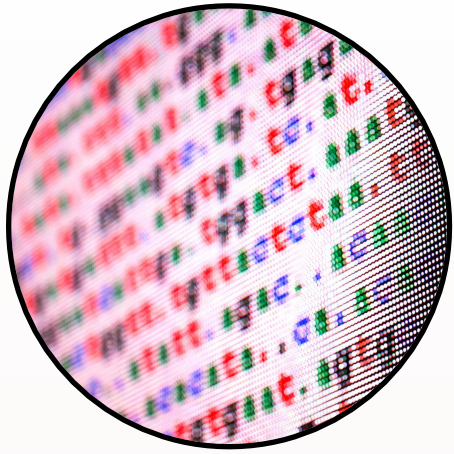
| Metacarácter | Ejemplo de uso                    | Resultado     | Descripción                    |
|--------------|-----------------------------------|---------------|--------------------------------|
| *            | Patrón: \d* sobre "123abc"        | ['123', '']   | 0 o más repeticiones           |
| +            | Patrón: \d+ sobre "Edad 45 años"  | ['45']        | 1 o más repeticiones           |
| ?            | Patrón: \d? sobre "A4C"           | ['', '4', ''] | 0 o 1 repetición               |
| {4}          | Patrón: \d{4} sobre "Año 2026"    | ['2026']      | Repite exactamente n veces     |
| {2,3}        | Patrón: [A-Z]{2,3} sobre "ABC XZ" | ['ABC', 'XZ'] | De 2 a 3 repeticiones          |
| [a-zA-Z]     | Patrón: [a-zA-Z]+ sobre "Hola123" | ['Hola']      | Letras mayúsculas o minúsculas |



# Expresiones regulares

## Cuantificadores y rangos

| Metacarácter | Ejemplo de uso                      | Resultado  | Descripción                                      |
|--------------|-------------------------------------|------------|--------------------------------------------------|
| (...)        | Patrón: (\d+) sobre "ID 345"        | ['345']    | Grupo de captura                                 |
| (?:...)      | Patrón: \d+(?:,\d+)? sobre "3,14"   | ['3,14']   | Grupo sin captura                                |
| ^            | Patrón: ^\d+ sobre "123 hola"       | ['123']    | Inicio de línea                                  |
| \$           | Patrón: \d+\$ sobre "hola 89"       | ['89']     | Final de línea                                   |
| (?=\d)       | Patrón: [a-z]+(?=\d) sobre "abc123" | ['abc']    | Lookahead positivo (lo que sigue debe coincidir) |
| (?<=\d)      | Patrón: (?<=\d)[A-Z]+ sobre "3AB"   | ['AB']     | Lookbehind positivo                              |
| (?<!\w)      | Patrón: (?<!\w)\d+ sobre "abc 12"   | ['12']     | Lookbehind negativo                              |
| `            | `                                   | Patrón: `A | B` sobre "B"                                     |



# Hands on

1. Extrae todos los números enteros de este texto: "Tengo 2 perros, 15 plantas y 300 fotos en el móvil."
2. Extrae todas las fechas de este texto: "Las reuniones son el 12/01/2026, 03/02/2026 y 7/3/2026."
3. Extrae las palabras formadas por letras: "Python3 es mejor que Java2, pero ambos son lenguajes."
4. Extrae emails del texto: "Mis correos son prueba@gmail.com y usuario@empresa.com."
5. Extrae los DNI del texto: "Clientes: 12345678A, 98765432Z, y un formato incorrecto 1234A."
6. Extrae los códigos alfanuméricos: "Los pedidos válidos son AB123, XY78 y ABC9999, pero no 123ABC"